

文件系统

存储是现代操作系统的重要功能之一。无论是日常生活中的图片、音乐、文档、视频、电子邮件，还是隐藏在大数据之下的账户信息、预测模型，均需要持久性地保存在计算机系统中。文件是最常用的存储抽象。文件系统是文件的管理者，利用存储设备可持久保存数据的能力，向上提供文件的抽象并实现访问文件的接口，是操作系统的重要组成部分。

存储设备的类型很多，包括磁盘、SSD、磁带等。为了简化文件系统的设计和实现，操作系统通常将存储设备抽象为一个以存储块（Block）为单位的大数组。存储块是存储设备的最小读写单元，其大小是固定的，常见大小为 512 字节或 4KB。也就是说，一个 4TB 的硬盘包含 10 亿个 4KB 的存储块。每个存储块通过一个地址进行标识和索引，该地址称为块号。文件系统的工作，就是将海量的存储块组织起来，向上提供文件的接口。至于如何将存储设备抽象为大数组，则由更底层的块层、设备驱动等负责，文件系统不用“操心”。

随着文件系统的发展，其自身逐渐进一步演化出不同的层次和模块。图 11.1 展示了 Linux 内核的存储软件栈。其中，在应用程序之下、块层之上的部分，均属于广义的文件系统，其内部进一步包括虚拟文件系统、具体的文件系统（如 Ext4、NTFS 等），以及页缓存模块。这些新的层次和模块沉淀了一些共性、通用的功能，使具体的文件系统可以聚焦在与文件组织及管理相关的逻辑上，也使新的文件系统更容易设计与实现。

本章首先以一个基于 inode 的文件系统为例，介绍文件系统的相关概念、

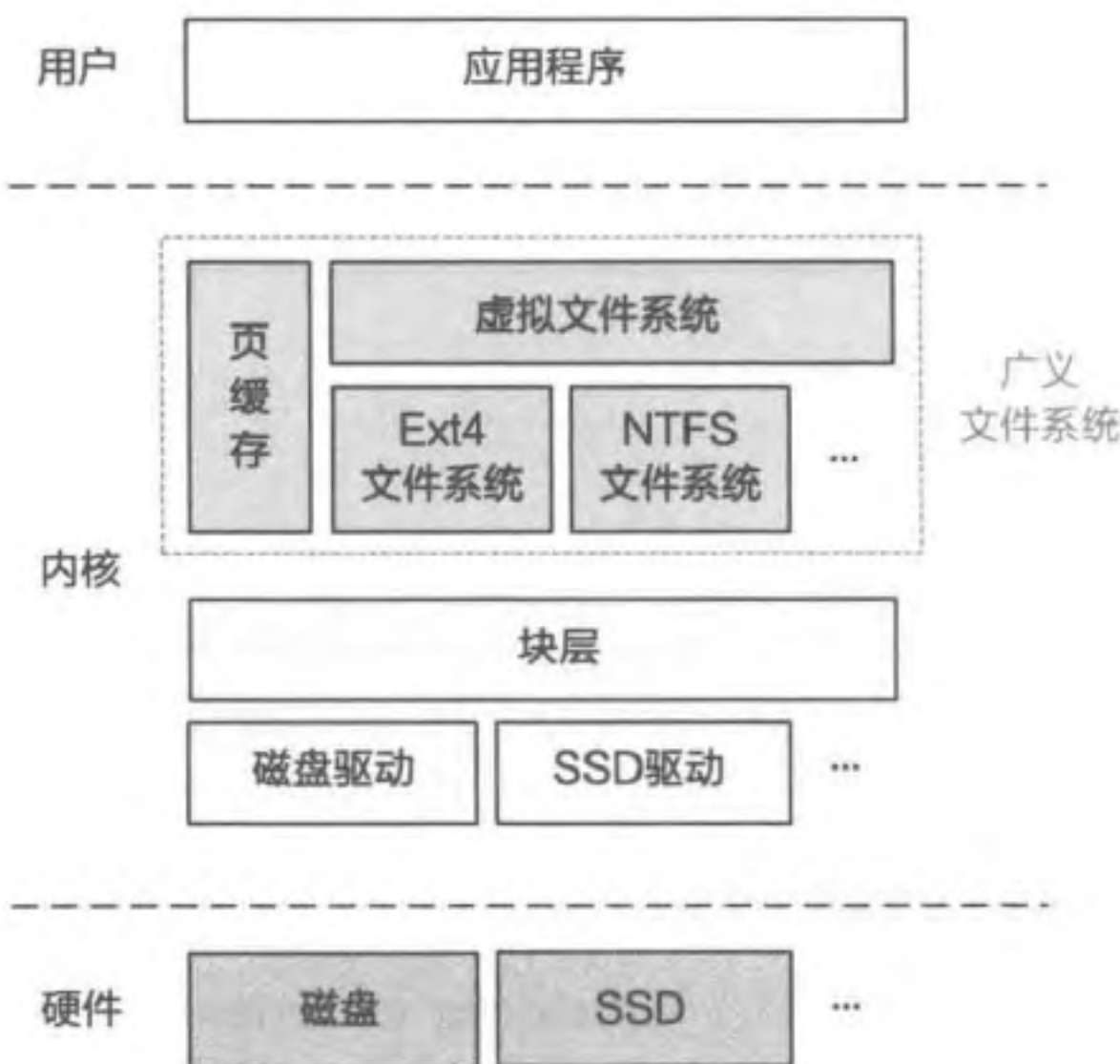


图 11.1 存储栈

常见设计和实现方法，并以不使用 inode 的 FAT 和 NTFS 两个文件系统作为补充。然后，以 Linux 中的虚拟文件系统为例，介绍操作系统如何通过将文件系统的内存部分（动态）与存储部分（静态）解耦，抽象出虚拟文件系统（VFS）层，实现对多个文件系统的统一管理。最后，介绍在微内核的思想下，FUSE 和 ChCore 如何实现用户态的文件系统。

11.1 基于 inode 的文件系统

本节主要知识点

- 基于 inode 的文件系统如何保存常规文件、目录文件和其他类型的文件？
- 对每种类型的文件有哪些操作，如何实现？
- 基于 inode 的文件系统如何规划使用存储空间？
- 什么是硬盘格式化？格式化后为什么硬盘的空间减少了？

当我们使用计算机时，不可避免地会使用或遇到过多种文件系统，其中许多文件系统（如 Ext4、BtrFS、F2FS）均基于 inode 概念而构建。在本节中，我们将介绍一个基于 inode 的文件系统的基本设计。为了叙述的简洁性，本节假设所有的存储结构均直接保存在块存储设备之上。后续的章节将对与这些存储结构相对应的内存缓存进行介绍。

11.1.1 一个不用 inode 的简单文件系统

在介绍 inode 的概念之前，我们首先为一个假想的场景设计一个简单的“文件系统”。在这个假想的场景中，用户只需要保存 10 个文件，文件名固定为从“0”到“9”，且每个文件的大小恰好为 4KB。由于每个文件的大小刚好是一个存储块，我们可以使用整个磁盘的 0 号存储块保存文件“0”对应的数据，使用 1 号存储块保存文件“1”对应的数据，以此类推（见图 11.2）。这个文件系统虽然简单，但已经支持对文件的查找和读写操作。文件的查找很直观——文件名就是磁盘块号，文件的读写则是对相应磁盘块的读写。当然，这个文件系统具有非常多的限制，文件的数量、文件名、文件大小都是固定的，很难用于真实场景中。下面将一步步去除这些条件限制，并随之更新文件系统的设计，以满足更加通用的使用场景。



图 11.2 针对假想场景的简单文件系统

随着用户业务量的提升，10 个文件已经不能满足用户的需求——用户需要保存更多的文件。同时，用户希望能够删除现有的文件，若读取已删除的文件，应返回“文件不存在”的提示信息。

我们此前的文件系统使用前 10 个存储块保存对应的文件数据。这种方式可以很方便地拓展到更多文件——假设我们使用一个容量为 1MB 的磁盘，最多可以保存 $1\text{MB}/4\text{KB} = 256$ 个文件。按照此前用户对文件的命名规则，我们的文件系统可以保存“0”到“255”这 256 个文件。若使用更大容量的磁盘，文件系统也自然可以支持更多文件。

在增加文件的数量后，剩下的一个问题是如何支持删除文件。为了能够正确地返回“文件不存在”，文件系统需要通过某种方式记录每个文件是否存在。一种直接的方式是为每个文件记录一位：若该位为 1，则表示对应的文件存在，允许正常访问；若该位为 0，则表示对应的文件不存在，若访问则返回“文件不存在”。在创建文件时，将该位设为 1；在删除文件时，将该位设为 0 即可。

由于这些位同样需要保存在磁盘中，因此文件系统不能再使用所有的磁盘存储块来保存文件数据。我们将根据整个磁盘的大小，将磁盘开头位置的若干存储块预留出来，专门用于保存这些位。假设使用一个容量为 1GB 的磁盘，共有 256K 个 4KB 的存储块。每个存储块对应一位，那么共需要使用 8 个存储块才足够保存这些位，即 $256 \times 1\,024 \times 1\text{b} = 256 \times 128\text{B} = 8 \times 4\text{KB}$ 。

现在，我们将整个磁盘的前 8 个存储块用于保存这些位，称为文件位图（Bitmap）。此后的第一个存储块用于保存文件“0”对应的文件数据，后面一个存储块用于保存文件“1”对应的文件数据，以此类推（见图 11.3）。在访问文件的时候，需要在文件名的数字上加 8 作为偏移量来得到存储块号，再访问磁盘得到数据。



图 11.3 带有位图的简单文件系统

11.1.2 inode 与文件

上述的场景对文件依然有一个非常大的限制——每个文件的大小必须恰好为 4KB。这在实际使用中并不现实，音乐、图片等文件的大小通常为数 MB，需要几千个 4KB 的存储块才能存下，而一个 4GB 的电影文件则需要 100 万个 4KB 的块。显然，此前使用的“一个存储块对应一个文件”的方法已经无法适用于这种场景。那么文件系统该如何记录并组织一个长度不固定的文件所对应的存储块呢？

inode 的提出就是为了解决这个问题。inode 是“index node”的简写，即“索引节

点”，用于记录一个文件的数据所在的存储块号（即索引）。此外，由于文件的大小不再固定为 4KB，因此 inode 还必须记录文件的大小。inode 中保存的信息，包括存储块号和文件大小，以及将来会介绍的更多文件属性（包括权限、修改时间、访问时间等），称为文件的元数据（Metadata）。

inode 同样需要保存在磁盘上。于是我们在磁盘头部再预留一块空间，以数组的方式连续保存所有的 inode，称为 inode 表。这样，我们可以用 inode 在表中的位置（即 inode 在数组中的下标）作为这个 inode 的标识，称为 inode 号。文件与 inode 是一一对应的，因此，只要得到一个文件的 inode 号，文件系统便可以访问该 inode 对应文件的所有数据与元数据。

现在我们需要对磁盘头部的预留区域做一些调整。原本的“文件位图”依然保留，但仅仅用于记录哪些磁盘块正在被使用，哪些块处于空闲状态，称为“存储块分配信息”区域。为了标记哪些 inode 正在被使用，哪些 inode 是空闲的，我们同样可以通过新增一块位图区域来表示，称为“inode 分配信息”区域。此外，需要标记两块位图与 inode 表的所在存储区域，因此我们将这些区域的起始块号和大小等信息单独记录在一个存储块中，称为超级块（Super Block），并规定文件系统开始的第一个块就是超级块，以方便加载。修改后的存储布局如图 11.4 所示。

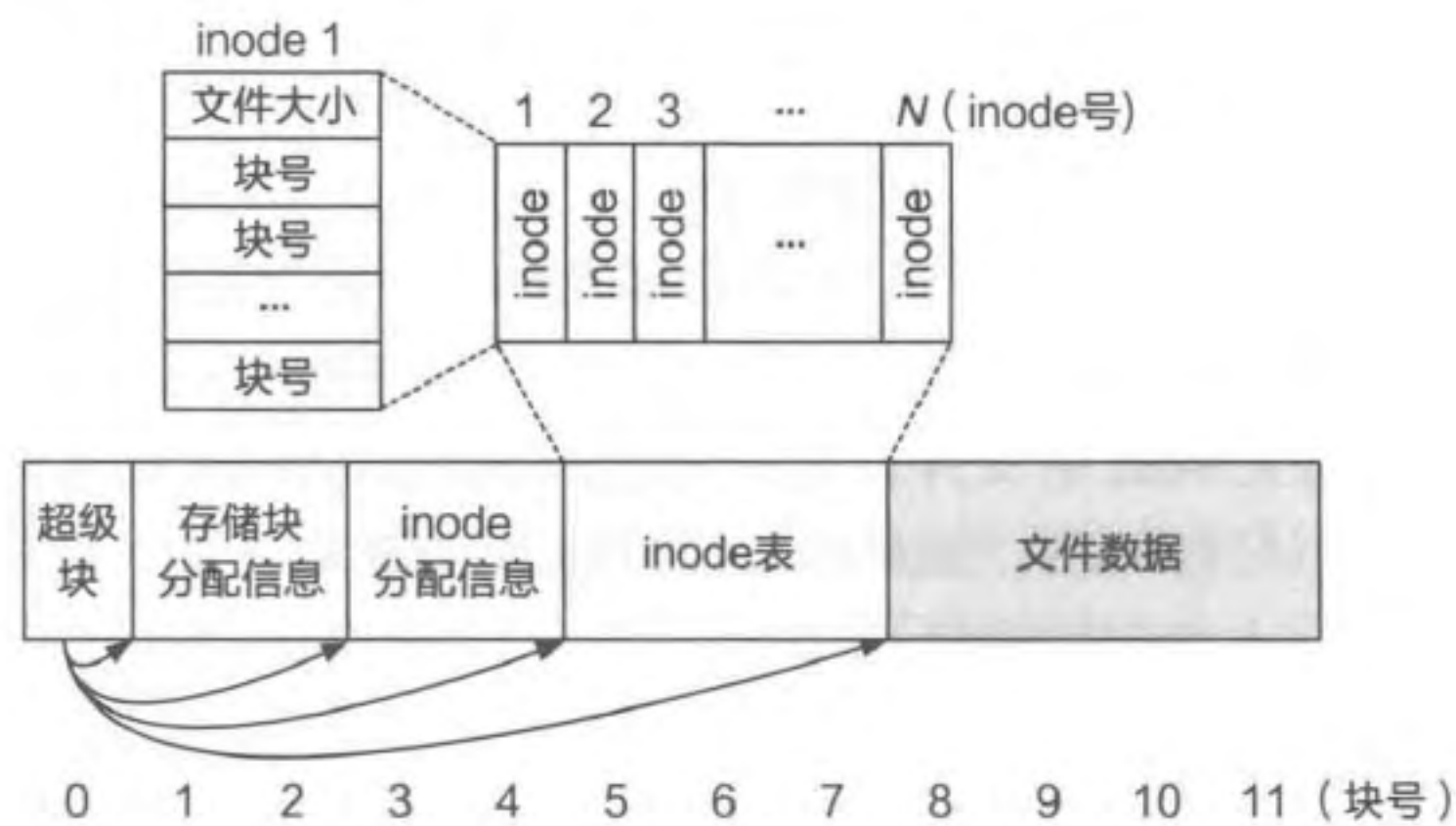


图 11.4 inode 文件系统的存储布局

到这里，一个基于 inode 的简单文件系统就已经基本成形了！在加载存储设备时，文件系统首先读取超级块，并通过超级块找到存储块分配信息、inode 分配信息、inode 表的位置，这样便完成了加载过程。之后，应用程序只需要给出 inode 号，文件系统便可以通过 inode 表找到对应的 inode 数据，并进一步找到该文件在存储设备中保存的所有数据。新建文件时，文件系统首先根据 inode 分配信息找到空闲的 inode，然后将 inode 分配信息中对应的位设置为 1；删除文件时，只需要将 inode 分配信息中对应的位设置为 0。现在的版本还有一个“小”缺点——不支持字符串文件名，只能用 inode 号来查找文件——我们后面会改进的。

在真实的文件系统中，inode 中除了记录存储的索引外，还记录了该文件相关的其他元数据，包括文件模式、文件链接数、文件拥有者和用户组、文件大小以及文件访问时间等（如表 11.1 所示）。其中，文件的模式表示不同的文件类型（如表 11.2 所示），包括常规文件、目录文件和符号链接文件等。不同类型的文件保存了不同种类的数据，在数据存储格式和使用方法上也会有所区别。在接下来的小节中，我们将对这些不同类型的文件及其设计进行逐一介绍。

表 11.1 POSIX 中定义的部分文件元数据

文件元数据	说 明
mode	文件模式，其中包括文件类型和文件权限
nlink	指向此文件的链接个数
uid	文件所属用户的 ID
gid	文件所属用户组的 ID
size	文件的大小
atime	文件数据最近访问时间
ctime	文件元数据最近修改时间
mtime	文件数据最近修改时间

表 11.2 Linux 中支持的文件类型

文件类型	文件用途
常规文件	保存数据
目录文件	表示和组织一组文件
符号链接文件	保存符号链接（指向目标文件的路径）
FIFO 文件	以队列形式传递数据，又称命名管道
套接字文件	用于传递数据，比 FIFO 文件更加灵活
字符设备文件	表示和访问字符设备
块设备文件	表示和访问块设备

11.1.3 多级 inode

在我们刚才设计的简单 inode 文件系统中，一个文件所有的存储块号顺序地保存在 inode 中，这意味着 inode 所需要的空间大小会随文件大小而变化。但由于文件系统无法提前获知每个文件所需要的块数量，其只能按照所支持的最大文件为每个 inode 预留空间。因此，为了支持最大 4GB 的文件，每个 inode 都不得不预留 8MB 的连续磁盘空间，哪怕实际上文件的大小只有 1KB。如果需要支持最大 4TB 的文件，则需要为每个 inode 预留 8GB 空间！这是巨大的空间浪费。

如何解决 inode 过大的问题？细心的读者可能觉得这个问题似曾相识：关于虚拟内存的一章中提到的单级页表同样有这个问题。页表与 inode 其实非常相似，页表的作用是将连续的虚拟地址映射到离散的物理地址，而 inode 的作用不也是把连续的文件偏移量映射到离散的存储块吗？那么，既然可以通过多级页表 + 动态分配页表的方法解决页表过大的问题，是否也可以通过多级 inode 来解决 inode 过大的问题呢？

答案是可以！我们可以采用分级的方式来实现 inode。默认的 inode 可以很小，当文件变大导致 inode 不够用时，文件系统可以从存储设备的数据区动态分配新的块作为 inode 的一部分，将块号记录在 inode 中（类似指针），从而使 inode 可以随文件同步增大。图 11.5 展示了一种简单且常见的数据块分级组织方式。inode 中保存了三种“存储指针”（即存储设备的块号）：第一种指针为直接指针，直接指向数据块，数据块中保存文件数据；第二种指针为间接指针，指向一个一级索引块，一级索引块中存放着指向数

据块的指针；第三种指针为二级间接指针，指向一个二级索引块，二级索引块中的每个指针均指向一个一级索引块，进而指向多个数据块。这些索引块都位于数据区，但实际上都属于元数据，可以将其看作 inode 的一部分。为了方便后面的讨论，我们之后均假设块大小为 4KB，每个指针占据 8 字节，每个 inode 中直接指针有 12 个，间接指针有 3 个，二级间接指针有 1 个。

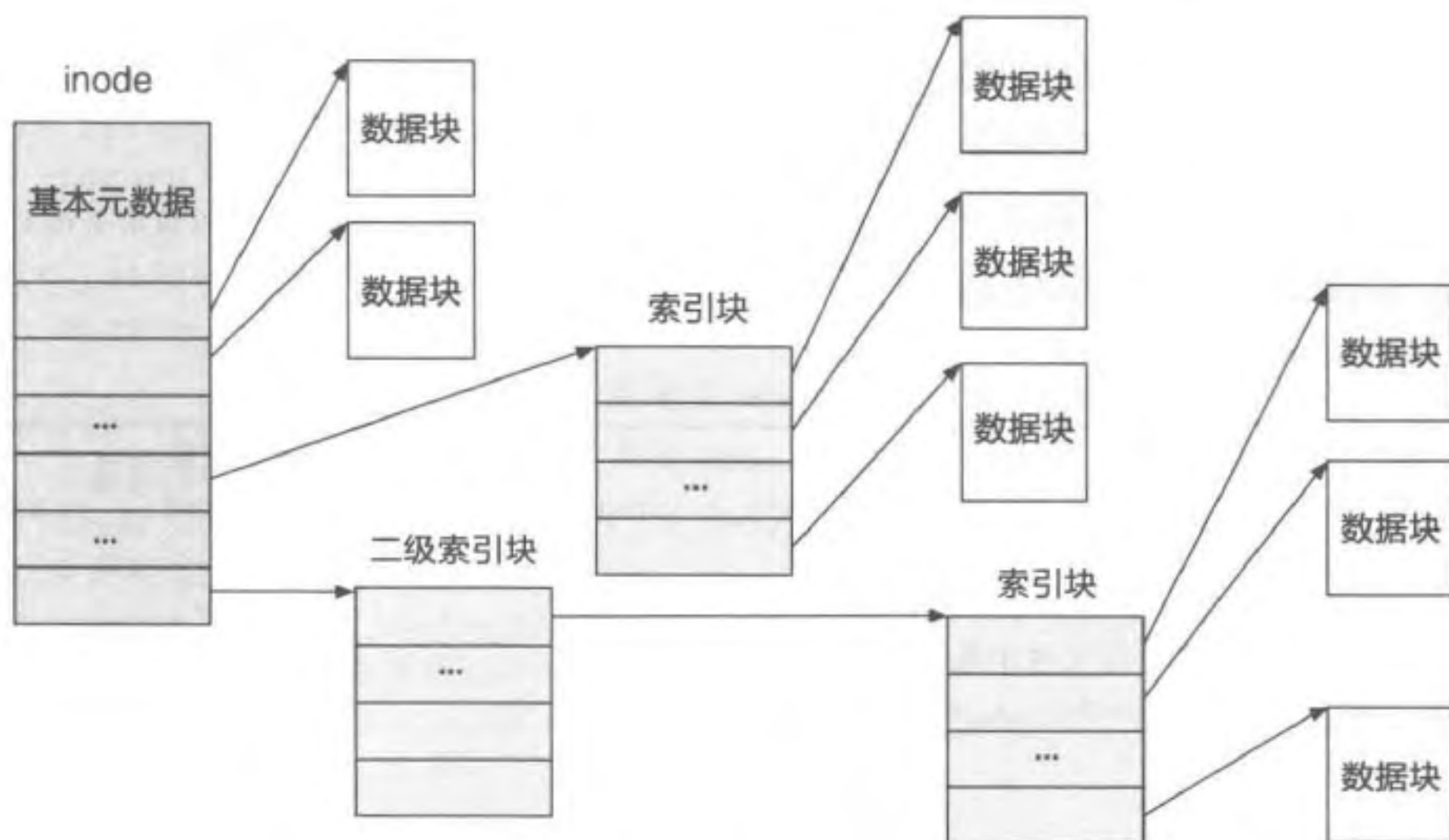


图 11.5 常规文件数据索引

在 inode 的第一个直接指针指向的数据块中，保存了整个文件最前端的 4KB 数据（即偏移量为 0 到 4 095B，包括 0 和 4 095）。第二个直接指针指向的数据块中保存了文件偏移量从 4 096 到 8 191 的数据。以此类推，最后一个直接指针指向的数据块保存了文件偏移量从 $4\,096 \times 11$ 到 $4\,096 \times 12 - 1$ 的数据。这些直接指针可以管理总共 48KB 的数据。如果一个文件的大小超过 48KB，则需要使用间接指针。

每个间接指针都指向一个一级索引块。一级索引块也占据 4KB 的空间大小，其中保存 512 个直接指针。因此，一个间接指针能够管理 512 个 4KB 的数据块，总共 2MB 数据。数据的定位方法与前述方法类似。在第一个间接指针指向的一级索引块中，第一个直接指针指向的数据块保存了这 2MB 数据中的前 4KB 数据，也就是整个文件的偏移量为 $4\,096 \times 12$ 到 $4\,096 \times 13 - 1$ 的 4 096B。在我们的假设场景下，inode 中的 3 个间接指针共管理了 6MB 数据。

当一个文件的大小超过 48KB+6MB 时，还需要用到二级间接指针。每个二级间接指针同样指向一个索引块，不过这个索引块是一个二级索引块，其中保存的 512 个指针为间接指针。换句话说，一个二级间接指针管理 512 个间接指针，因此可以管理 $2\text{MB} \times 512 = 1\text{GB}$ 数据。此时一个 inode 所能管理的最大文件大小为 48KB+6MB+1GB。

从上述组织方式中可以看出，一个文件系统所支持的最大文件大小受文件数据组织

方式的限制^①。因此，可以通过调整 inode 的结构来改变其能管理的最大文件大小。例如，为了管理更大的文件，inode 中的索引可以使用更多的二级间接指针，甚至启用三级或者四级间接指针。

代码片段 11.1 和代码片段 11.2 中给出了简单 inode 文件系统中文件读写的大致流程。文件的读取和写入流程十分相似，总体来说，可以分为以下步骤：

1. 进行参数的基本检查。
2. 根据 offset 从 inode 中找到对应块的块号。
3. 从（向）块号对应的数据块中读（写）数据。
4. 重复步骤 2 和 3 直至满足用户指定的数据大小。
5. 更新 inode 中的时间等元数据。

代码片段 11.1 简单 inode 文件系统中的文件写入流程

```

1 // 将 buff 中长度为 size 字节的数据写入 inode 对应文件的 offset 开始的位置
2 int write(struct inode *inode, off_t offset, const char *buff, size_t size)
3 {
4     size_t total_written = 0;
5     // 不允许写入位置超过文件末尾
6     if (offset > inode->i_size)
7         return 0;
8
9     while (size > 0) {
10         // 获取对应位置的 block 号
11         long block_id = get_block_id_from_inode(inode, offset);
12         if (block_id == NO_BLOCK) {
13             // 如果没有对应的 block，则分配新的 block 并加入 inode 中
14             block_id = allocate_block();
15             add_block_id_to_inode(inode, offset, block_id);
16         }
17         // 将数据写入 block 中
18         size_t nr_written = write_data_to_block(block_id, offset, buff, size);
19         // 更新对应的计数，为下一个 block 的写入做准备
20         offset += nr_written;
21         size -= nr_written;
22         buff += nr_written;
23         total_written += nr_written;
24     }
25     // 如果写入将文件扩大了，则更新文件大小
26     if (offset > inode->i_size)
27         inode->i_size = offset;
28     // 更新 inode 中维护的各类时间
29     update_inode_time(inode);
30     // 返回总共写入的字节数，此处我们省略了一些异常情况

```

① 一些其他因素同样会限制最大文件大小。例如，在 FAT32 文件系统中，“文件大小”这项元数据只有 4 字节，导致一个文件无法超过 2^{32} 字节，即 4GB。

```

31 return total_written;
32 }

```

代码片段 11.2 简单 inode 文件系统中的文件读取流程

```

1 // 从 inode 对应文件的 offset 位置开始读取 size 字节到 buff 中
2 int read(struct inode *inode, off_t offset, char *buff, size_t size)
3 {
4     size_t total_read = 0;
5     // 不允许读取开始位置超过文件末尾
6     if (offset > inode->i_size)
7         return 0;
8     // 如果读取结尾位置超过文件末尾, 则将读取量减少
9     if (offset + size > inode->i_size)
10         size = inode->i_size - offset;
11
12     while (size > 0) {
13         // 获取对应位置的 block 号
14         long block_id = get_block_id_from_inode(inode, offset);
15         // 所有 block 都应存在
16         assert (block_id != NO_BLOCK);
17         // 读取对应位置的数据到 buff
18         size_t nr_read = read_data_from_block(block_id, offset, buff, size);
19         // 更新对应的计数, 为下一个 block 的读取做准备
20         offset += nr_read;
21         size -= nr_read;
22         buff += nr_read;
23         total_read += nr_read;
24     }
25     // 更新 inode 中维护的各类时间
26     update_inode_time(inode);
27     // 返回总共读取的字节数, 此处我们省略了一些异常情况
28     return total_read;
29 }

```

在上述代码中, 我们可以发现 `offset` 被多次传入不同的函数中。在 `get_block_id_from_inode` 和 `add_block_id_to_inode` 中, `offset` 用于计算对应位置在文件的第几个数据块中, 从而进一步根据多级 `inode` 结构获取块号。在 `write_data_to_block` 和 `read_data_from_block` 中, `offset` 用于计算读写位置在数据块内部的偏移量。

我们还可以发现读写操作在一些细节处理上有所不同。例如, 对于文件写入操作, 如果对应的块不存在, 那么需要首先分配数据块, 并将其块号加入 `inode` 结构中, 再进行对该数据块的写入; 如果写入后文件的大小变大, 则还需更新 `inode` 结构中记录的文件大小。对于读取操作, 如果读取的结尾位置超过文件当前大小, 则会减少读取的数据量, 只读取到当前的文件结尾。上述代码给出的是简单 `inode` 文件系统的一种实现方法, 不同文件系统对于这些情况的具体处理会有所不同。另外, 上述代码中省略了一些其他的检查, 例如对数字的溢出检查, 读写磁盘时的 I/O 错误处理等, 在实际实现中, 这些

检查也是非常重要的。

11.1.4 文件名与目录

前文提到，文件与 inode 是一一对应的，对于计算机程序来说，通过 inode 号就可以找到对应的文件。然而对于用户来说，使用 inode 号作为文件名并不合适。除了难以记忆之外，使用 inode 号直接表示文件还造成了 inode 号与文件存储位置的强耦合。由于 inode 号与文件 inode 结构的存储位置一一对应，文件系统无法在不改变 inode 号的情况下更改文件 inode 的存储位置，也无法用一个新的 inode 号指代一个已有的 inode 结构。

为了对用户更加友好，我们需要在文件系统中加入字符串形式的文件名，增加一层从文件名字符串到 inode 号之间的映射。字符串形式的文件名不但更方便记忆，也使文件名与具体的存储位置解耦。细心的读者可能已经发现，表 11.1 中的 inode 字段并不包含文件名，换句话说，在我们介绍的 inode 文件系统中，文件名并不是文件的元数据。那么，文件名与 inode 的映射保存在哪里呢？答案是目录。让我们把书暂时翻到目录部分仔细观察，一本书的目录，不正是把一个字符串（章节标题）映射到一个编号（页码）吗？

接下来，我们应当如何改造文件系统，使其支持目录功能呢？一种直接的想法是：在存储设备的头部再预留一块空间，专门用于存放目录的内容，就像书的目录总是放在开头一样。这种设计是可行的，但会带来三个问题。第一，该预留多少空间呢？文件名是可以修改的，长度也是可以变化的，如果为每个 inode 都预留足够多的空间，又会导致空间的浪费。第二，如何根据文件名找到对应的 inode 呢？如果只是按照 inode 的顺序来存储文件名，意味着文件名本身是乱序的，只能通过遍历所有文件名的方式进行查找，这显然是非常慢的。第三，目录与 inode 是并列的，这意味着 inode 需要的数据结构和处理逻辑，目录同样需要一份。例如，同样需要增加一块预留空间，记录哪些目录是空闲的，哪些是被使用的。

可以看到，目录需要保存一定的数据，数据需要允许读取和修改，且数据的内容会动态增加或减少——这些需求不正是 inode 能够提供的吗？于是，一个“偷懒”的想法产生了：将目录看作一种特殊类型的文件，复用之前介绍的 inode 机制来实现目录！换句话说，将目录的抽象建立在 inode 的基础之上，使目录的存储方式、查找过程和访问逻辑与普通文件一样。唯一的区别在于，目录文件的格式是由文件系统规定的，数据的读写是由文件系统控制的，而不像普通文件那样完全由应用程序控制。这种复用大大简化了文件系统的设计和实现，之前的存储布局不需要做任何改动，只需要在文件系统的代码逻辑中增加一个 inode 类型即可（记录在表 11.1 中的 mode 字段）。

在我们的 inode 文件系统中，目录的内容（也就是目录文件的数据）就是一条条从文件名到 inode 号的映射，每条映射称为一个目录项（Directory Entry, dentry）。图 11.6 展示了一个目录项内的结构，包括文件名、文件名的长度、文件名对应的 inode 号和整个目录项的长度^①。目录项长度用于标记整个目录项的长度，主要为目录项的删除和重用而

^① 在一些文件系统中，目录项中还会保存目标文件的类型，以避免进行目录遍历时访问目标 inode。

设计。文件名长度则顾名思义，记录着后面保存的文件名的有效长度。以此格式，目录项一个接一个地存放在文件的数据块中，组成一个连续的字符序列，从而可以直接复用常规文件的数据组织方式。对目录的操作主要包括查找、遍历、删除和新增，具体实现如下。

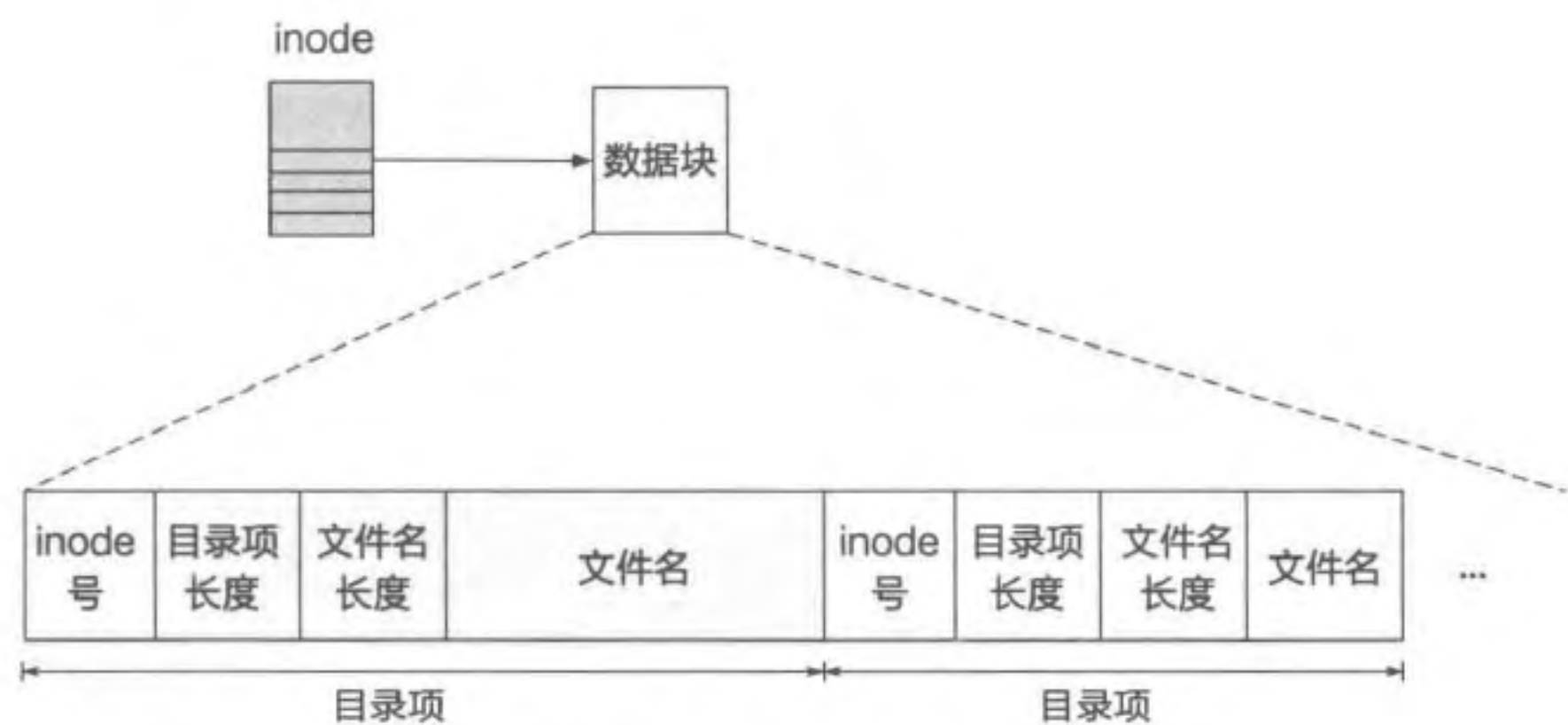


图 11.6 目录文件中的内容：目录项

- 在目录中查找文件：在目录中查找某个文件名时，文件系统从目录文件的数据块中记录的第一个目录项开始依次比较，当某个目录项中的文件名与要查找的文件名相同时，返回对应的 inode 号。由于只需要查找当前目录保存的目录项，所以查找时间远小于遍历文件系统 inode 表的时间。
- 遍历目录：与查找文件名的过程类似，文件系统找到该目录文件的数据块，依次检查保存的所有有效（还未被删除）的目录项，并返回结果。
- 删除目录项：文件系统首先在该目录中找到这个文件，然后从目录中删除对应的目录项。一种常见的实现方法是将目录项中的 inode 号设为 0，表示该目录项是无效的。这种方法无须为目录有效性信息预留额外的存储空间，可以更高效地利用空间。同时，删除后还可以将相邻的无效目录项进行合并，以允许更长的新目录项重新利用这些空间。
- 新增目录项：在目录中创建新文件时，文件系统会在该目录中增加一个新的目录项，记录新文件的文件名和对应的 inode 号。为优化存储空间，如果目录文件中有无效（已被删除）的目录项，且该目录项的空间足够保存新文件的文件名，则可以重用此目录项的位置存放新的目录项。如果找不到合适的无效目录项，则新的目录项会以追加的方式记录在目录文件末尾。

代码片段 11.3 中给出了简单 inode 文件系统中创建一个目录文件的大致流程。文件系统首先进行一些必要的检查，如文件名的合法性检查。不同文件系统对于文件名的合法性有不同的要求，在本章的简单 inode 文件系统中，文件名不能为“.”或者“..”，不应包含“/”字符，且长度不应超过 255 字节。该检查逻辑在 `is_filename_valid` 函

数中实现。同时，一个目录中不应存在同名文件，因此还需要检查将要创建的文件名是否已经存在（`name_exists` 函数），这部分的实现方式与前文中的“在目录中查找文件”相同。

代码片段 11.3 简单 inode 文件系统中的目录文件创建流程

```

1 // 在 dir 对应的目录中添加名为 name 的目录文件
2 int mkdir(struct inode *dir, const char *name, size_t namelen)
3 {
4     // 检查文件名的合法性（例如不能包含特殊字符）
5     if (!is_filename_valid(name, namelen))
6         return -ENAME;
7     // 检查该文件名是否已经在 dir 中存在
8     if (name_exists(dir, name, namelen))
9         return -EEXIST;
10    // 创建新的 inode
11    struct inode *inode = create_dir_inode();
12    // 将文件名和 inode 的对应关系作为目录项写入 dir
13    err = add_dir_entry(dir, name, namelen, inode->i_no);
14    if (err) {
15        // 如果出错，需要释放此前分配的 inode
16        delete_dir_inode(inode);
17        return err;
18    }
19    // 更新 dir 中的链接数
20    update_dir_nlink(dir);
21    // 更新时间
22    update_inode_time(dir);
23    return 0;
24 }
```

在这些检查之后，文件系统为新目录创建并初始化一个新的 inode（`create_dir_inode` 函数），之后将文件名和该 inode 的对应关系作为新的目录项添加到父目录（`dir`）中。正如前文所述，在添加目录项时，文件系统首先为该新目录项寻找或者分配存储空间，再将目录项信息写入。这些步骤在 `add_dir_entry` 函数中实现。

如果一切顺利，还需要更新父目录中记录的链接数和时间等信息。文件的链接数将在 11.1.6 节进一步介绍。如果添加目录项时发生了错误，则需要将 inode 销毁并释放（`delete_dir_inode` 函数），同时返回错误信息。

由于目录本身也是文件，因此可以通过递归的方式层次化地组织文件系统中的所有文件。换句话说，目录与普通文件一样也有文件名，其文件名保存在上一层目录文件中；上一层目录的文件名则保存在更上一层目录文件中，一直递归到最高一层目录——根目录。根目录没有文件名，因为已经没有更上一层目录了。文件系统并不通过文件名来找到根目录，而是将第一个 inode 作为根目录文件。

从用户的角度来看，整个文件系统是以根目录为根节点的树状结构。除了根目录外，

每个文件和目录都在更高层的目录之下。文件系统通常以文件路径来确定一个文件，文件路径是一个字符串，其中包含若干由“/”隔开的文件名^①。当给定一个文件路径来查找对应的 inode 时，文件系统首先从根目录中查找第一级目录，再从第一级目录中查找第二级目录，不断迭代，最终查找到最后一级的文件名，并在目录项中找到 inode。

小知识 根目录没有文件名

在 UNIX/Linux 系统中，使用“/”表示根目录，例如“ls /”命令。那么，根目录的文件名不是“/”吗？为什么说根目录没有文件名呢？其实，“/”只是路径的分隔符，用于分隔不同层次的目录和文件，比如“/home/mospi/os-book.pdf”。使用“/”表示根目录仅仅是一种约定，根目录真正的文件名应该是在“/”之前，也就是——没有。

练习

- 11.1 假设文件系统中有两个目录。其中一个保存 10 个电影文件 (movie-1.mp4 ~ movie-10.mp4)，另一个保存 100 个笔记文件 (note-1.md ~ note-100.md)，哪个目录文件占的存储空间更大？

11.1.5 存储布局

在加入 inode 和目录支持之后，简单 inode 文件系统的存储布局已经比较完整了，如图 11.7 所示。可以看到，存储设备目前被划分成超级块、块分配信息、inode 分配信息、inode 表和数据区这 5 个部分。当在一个存储设备上创建新的文件系统时，文件系统格式化工具会根据文件系统的存储布局和存储设备的容量，计算每个区域的大小，并初始化区域中的元数据，这个过程也被称为格式化操作。下面我们对图 11.7 中的各个区域分别进行回顾。

在我们的简单 inode 文件系统中，超级块位于文件系统的第一个存储块，负责记录整个文件系统的全局信息，主要包括其他区域在存储中的位置，即每个区域开始的存储块号。在真实的文件系统中，超级块通常还会包含用于标识文件系统类型的魔数、版本信息、存储的空间大小、能支持的最大 inode 数量、当前空闲可用的 inode 的数量、能支持的最大的块数量、当前空闲可用的块数量，以及一些统计信息，如最近一次的挂载时间等。同时，由于超级块中包含的信息非常重要，通常会在存储设备的不同区域保存多个备份。在超级块之后，是块分配信息与 inode 分配信息这两个位图区域，分别记录

^① Windows 使用“\”字符作为路径的分隔符。

数据区的存储块与 inode 的使用情况。分配信息区域后是 inode 表，保存所有的 inode 结构。通常来说，由于 inode 表的区域不能动态调整，因此文件系统所能保存的最大文件数量在格式化的时候就已经确定了。在 inode 表之后的存储空间为数据区。需要注意的是，数据区不仅保存普通文件的数据，还会保存 inode 的索引块，以及目录的数据——别忘了目录也是一种文件。

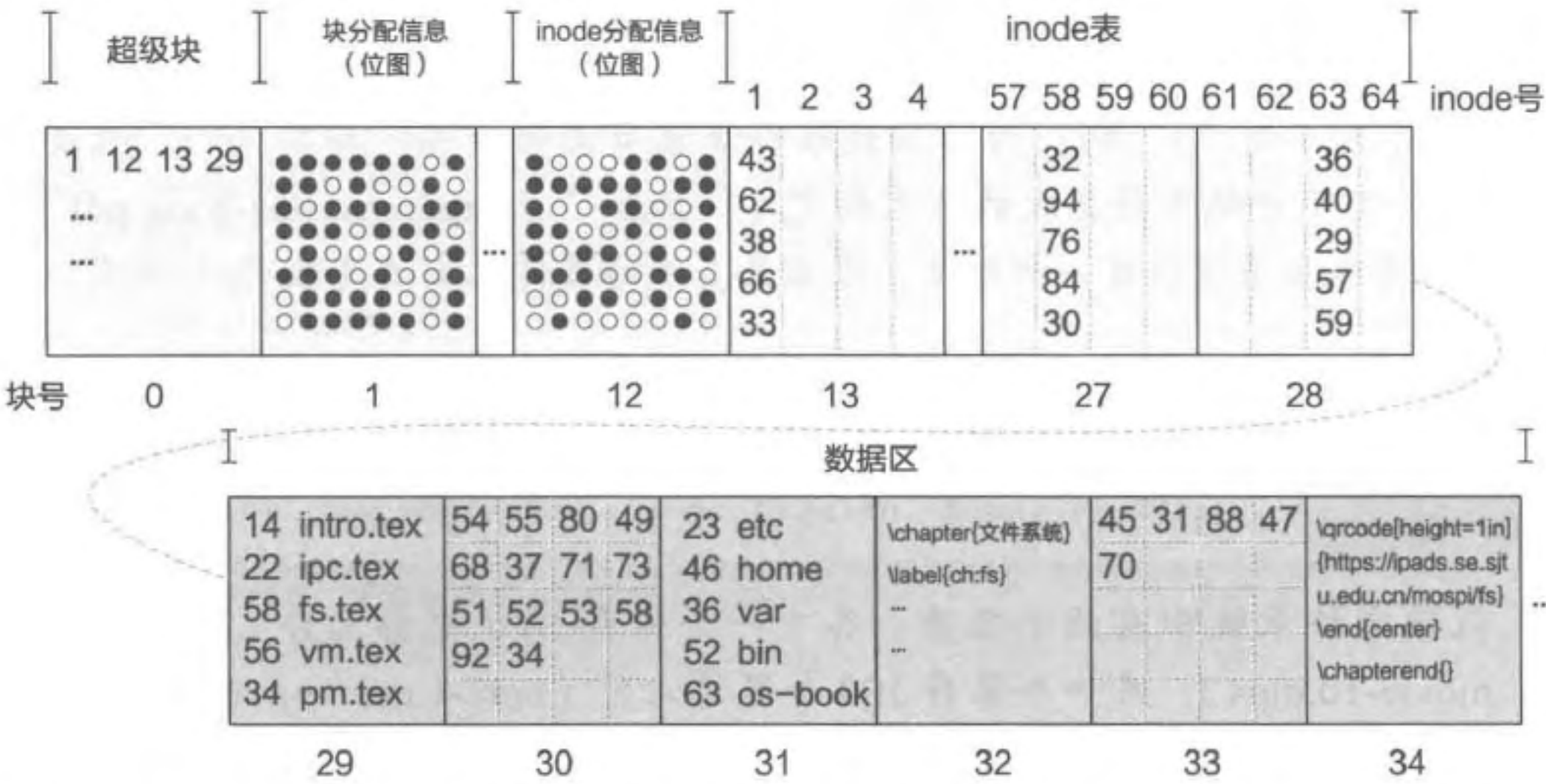


图 11.7 文件系统的存储布局和内容示例。图中的●表示 1 (使用中)，○表示 0 (空闲)。inode 仅标出了 4 个直接块号和 1 个索引块号 (一级索引)

小知识：为什么在格式化磁盘后，系统显示的可用空间变小了？

通常来说，操作系统显示的磁盘可用空间主要是数据区的大小，不包括数据块之前的预留区域。对于一个容量为 4TB、块大小为 4KB 的硬盘，仅块分配信息的区域就要占用 128MB。假设每 16KB 数据对应一个 inode，每个 inode 大小为 256B，则 inode 表大小为 64GB，占了整个硬盘的 1.6%。

练习

11.2 请根据图 11.7，描述文件系统是如何查找到 “/os-book/fs.tex” 所对应的 inode，并读取其文件数据的。

11.1.6 从文件名到链接

通过目录来记录文件名与 inode 号的对应，而不是将文件名作为 inode 的一部分，这种实现方式带来了一个非常有意思的功能：链接。由于一个文件的文件名与这个文件

的 inode 并没有直接绑定，而是通过目录项实现两者间的映射，因此我们完全可以为一个 inode 建立多个映射——换句话说，一个文件可以有多个文件名。文件系统提供了一个专门的系统调用，为一个已经存在的文件建立新的文件名映射，这种映射就是链接 (Link)。链接也称“硬链接”，这是因为后来出现了一类新的“符号链接”，为了更好地区分，因此将原有的链接称为“硬链接”，将新出现的符号链接称为“软链接”。符号链接 / 软链接将在后文介绍，本节所指的链接均为硬链接。

文件名与链接

链接和文件名有什么区别呢？从文件系统的角度来看，两者没有任何区别，都是保存在目录项中，映射到一个 inode 号，因此所有的文件名都是链接。但出于习惯，我们通常会将文件创建时使用的字符串称为该文件的文件名，而将通过链接命令创建的文件名称为链接。

为一个文件新建链接的方法非常简单：以一个字符串和 inode 号作为参数，新建一个目录项即可。目录项可以记录在任意目录中，前提是新的字符串与该目录中其他目录项的文件名不重复。但是，在删除链接时却遇到了一个问题：删除链接所在的目录项时，我们是否应该也删除这个文件的 inode 和数据呢？如果这个链接是指向该文件的最后一个文件名，那么应当也删除该文件本身；但如果这个文件还有其他的文件名 / 链接，则不能删除文件数据。那么，我们该如何知道当前被删除的链接是不是该 inode 的最后一个文件名呢？

为了准确地记录何时能够将 inode 和其对应的文件从文件系统中删除，文件系统在 inode 中增加了一个引用计数器，也称链接数（即表 11.1 中的 nlink），记录有多少个链接指向了该 inode。每当一个目录项被加入目录中，目录项中对应的 inode 的链接数就会增加 1；每当一个目录项被删除，目录项中对应的 inode 链接数就减少 1；当 inode 的链接数减为 0 时，该 inode 和对应的文件可以被销毁和回收。

使用链接可以很方便地实现类似 Windows “快捷方式”的功能。例如，一个文件被保存在层次很深的目录中，每次使用都十分烦琐，而通过链接，用户可以在其他目录中为该文件起一个简单的别名，以方便访问。输出记录 11.1 中用 `ln` 命令为文件“a”创建了一个新的链接“b”，所有对文件“b”的操作均是对文件“a”的操作，因为两者的 inode 是一样的（通过 `ls -li` 可以显示文件名对应的 inode 号）。

输出记录 11.1 使用 `ln` 命令创建硬链接

```
[user@osbook ~] $ touch a
[user@osbook ~] $ ln a b
[user@osbook ~] $ echo "hello, world" > b
[user@osbook ~] $ cat a
```



```
hello, world
[user@osbook ~] $ ls -i
90050313 a      90050313 b
[user@osbook ~] $ mkdir d
[user@osbook ~] $ ln d e
ln: d: Is a directory
```

需要注意的是，在大多数文件系统的实现中，链接的目标文件不能是目录文件。这是因为如果允许为目录创建链接，很容易形成环，从而导致错误的情况。让我们来看下面一个例子：有一个 `/a/` 目录，其 `inode` 号为 100。现在创建一个链接 `/a/a-link` 指向 `/a/` 目录对应的 `inode`（即 `inode-100`）。此时，会出现无限深的路径 `/a/a-link/a-link/a-link/...`。另外，任何在 `/a/` 下的文件也都可以通过 `/a/a-link/` 来访问——这往往会给人一种错觉，以为这些文件是不同的。对于用户来说，“目录是树状的”这一理念已经深入人心，一旦允许目录出现环状结构，有可能会使用户更容易犯错^①。因此，大部分文件系统都不允许为目录创建链接，除了两个例外——“.”和“..”是指向目录的链接。

小知识：目录的链接数

在 POSIX 中，每个目录均包含两个特殊的目录项“.”和“..”，其对应的 `inode` 号分别为该目录本身和其父目录。这两个目录项由文件系统进行管理，与目录文件一同被创建和删除。对于根目录来说，“.”和“..”对应的 `inode` 号均为根目录自身。由于目录文件中保存了“.”和“..”两个目录项，当创建一个新的目录时，其本身的链接数为 2，同时其父目录的链接数需要加 1。

链接机制存在一个限制：为一个文件新建的链接必须和这个文件在同一个文件系统中。虽然我们还没有学习多个文件系统如何在一个文件树下同时存在（将在 11.3.3 节介绍），但依然可以凭经验设想下面的场景：在一台 Ubuntu Linux 主机中插入 U 盘（假设 U 盘也使用 `inode` 文件系统），一般会默认把 U 盘的文件系统挂载到“`/media/`”目录，假设是“`/media/usb`”目录。于是，用户可以通过“`/media/usb/a.txt`”这个路径来访问位于 U 盘根目录的文件“`a.txt`”。然而，如果用户希望在其他目录——比如“`/os-book`”——创建一个到“`/media/usb/a.txt`”的链接，则会失败。这是因为 U 盘文件系统与主机文件系统是不同的，其 `inode` 表也是不同的，U 盘的 100 号 `inode` 与主机的 100 号 `inode` 是两个完全不同的文件。所以，如果一定要在主机文件系统的某个目录创建一个链接指向 U 盘的 100 号 `inode`，那么在访问这个链接时，只会错误地访问到主机文件系统的 100 号 `inode`。那么，如何才能实现跨文件系统的链接呢？我们需要引入一种新的链接方式：符号链接。

① 在支持了链接之后，文件系统已经不是严格的树状结构了。

11.1.7 符号链接（软链接）

符号链接（Symbolic Link）是一种建立在文件路径层次之上的链接。符号链接文件本身是一个独立的文件，具有自己的文件名和 inode。但与普通文件不同，符号链接是一种特殊的文件^①，其数据内容是一个字符串，即该链接所指向的目标文件的路径。为了与传统的基于 inode 号的链接更好地区分，人们一般也将符号链接称为软链接（Soft Link），而将之前的链接称为硬链接（Hard Link）。

新建软链接时，用户仅仅传入一个字符串，操作系统会新建一个类型为软链接的 inode，并将字符串作为其数据。由于路径字符串的长度一般不会很长，因此文件系统可以做一个简单的优化：将路径字符串直接保存在 inode 中，占据原本用于保存数据块指针的空间，这样就可以不使用任何数据块，从而节约存储空间^②。当用户打开一个软链接文件时，文件系统会根据 inode 类型判断出这是一个软链接，于是会读取其保存的路径字符串，然后以该路径为参数再次查找对应的 inode，若没有找到则报错。因此，用户打开软链接文件，最终会打开其所指向的目标文件。从用户的角度来看，这个效果和硬链接是非常类似的。与硬链接不同的是，软链接和目标文件是两个不同的、独立的文件，可以保存在不同的文件系统中，因此可以实现跨文件系统的链接。

输出记录 11.2 展示了软链接的创建和其他操作示例。用户可通过带 `-s` 参数的 `ln` 命令创建一个软链接，其中 `-s` 后的字符串便是目标文件的路径。软链接和目标文件是独立的，拥有各自的 inode，因此即使目标文件不存在，软链接也可以存在，只是访问的时候会报错。软链接可以指向目录，这也是其与硬链接的不同之处。软链接的目标文件可以是另一个软链接，文件系统只需要再次根据路径查找即可。为避免出现回环导致无限深的路径，文件系统需要为路径的深度设置一个阈值，若超过该阈值则报错。

输出记录 11.2 使用 `ln -s` 命令创建符号链接

```
[user@osbook ~] $ ln -s a a.link
[user@osbook ~] $ cat a.link
cat: a.link: No such file or directory
[user@osbook ~] $ echo "hello, world" > a
[user@osbook ~] $ cat a.link
hello, world
[user@osbook ~] $ ls -l
-rw-r--r-- 1 osbook wheel 13 6 12 19:45 a
lrwxr-xr-x 1 osbook wheel 16 12 19:45 a.link -> a
[user@osbook ~] $ ls -li
90055602 a          90055547 a.link
[user@osbook ~] $ mkdir d
[user@osbook ~] $ ln -s d d.link
[user@osbook ~] $ touch d/x
```

① 回顾一下：目录文件也是一种特殊的文件，文件的类型记录在该文件 inode 的 `mode` 字段。

② 这个优化不仅可以用于软链接，而且可以用于其他很小的文件。


```
[user@osbook ~] $ ls d.link
x
[user@osbook ~] $ ln -s e e
[user@osbook ~] $ cd e
cd: too many levels of symbolic links: e
```

文件系统的设计空间非常广，inode 仅仅是其中的一种，现实中还有许多并非基于 inode 的文件系统。那么，不同的设计会为文件系统带来哪些能力上的不同呢？接下来我们介绍 Windows 系统中常用的两个文件系统——FAT 和 NTFS 的设计。

11.2 基于表的文件系统

本节主要知识点

- ❑ FAT 文件系统是如何保存数据的？
- ❑ NTFS 与 FAT 文件系统相比有哪些改变，带来了哪些好处？
- ❑ 用户态文件系统有哪些优势？如何在宏内核中提供用户态文件系统机制？

11.2.1 FAT 文件系统

文件分配表（File Allocation Table，FAT）^[14]是一种组织文件系统的架构。1977 年，Bill Gates 和 Marc McDonald 创建了第一个基于 FAT 的文件系统，用于管理 Microsoft Disk BASIC 系统中的存储设备^[5]。自被搭载在 Windows 操作系统中至今，基于 FAT 的文件系统从早期的 FAT12、FAT16 逐步发展到被广泛使用至今的 FAT32 和 exFAT 等文件系统。这些文件系统均使用 FAT 作为主要索引格式，后文中我们统一使用 FAT 文件系统来表示这一类文件系统。本节中，我们将以 FAT32 为基础介绍 FAT 文件系统的基本设计[Ⓐ]。

存储布局

图 11.8 展示了 FAT 文件系统的基本存储布局。位于整个文件系统最前端的是引导记录。引导记录的作用和超级块类似，其中记录了文件系统的元数据以及后续各个区域的位置。若此分区是可启动分区，则引导记录中还包括引导整个操作系统启动所需要的代码。

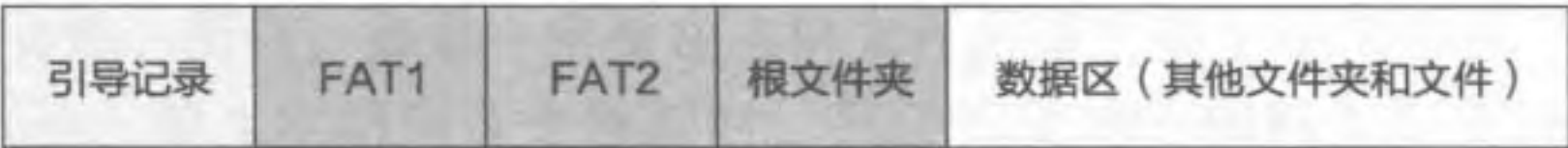


图 11.8 FAT 文件系统的存储布局^[2-3]

Ⓐ 由于 FAT 文件系统的版本众多，且在设计上会略有差别，实际使用中的 FAT 文件系统设计可能与本节的内容略有出入。